INTRODUCTION TO NETWORK SYSTEMS · CLASS 2 (VIDEO)

Where does this frame start?

Framing and error detection at the link layer

Dr. Thananop Thongthavorn
School of Information Technology, KMITL

WHAT ARRIVES AT THE NETWORK CARD

0	1	1	0	1	0	0	1	1	1	0
1	0	1	0	0	0	1	1	0	1	0

Two questions for today

1. Where does one frame end and the next begin?
2. Did any bit change on the way?

How to use this video

Watch it before Class 3 (live)



Watch in order

Seven clips of 8-16 minutes. Take a break between clips; do not watch it all at once.



Pause and try

When you see the orange pause sign, stop the video and work it out on paper.



Two things to hand in

A short worksheet (attendance) and a longer practice set (exam preparation). Both before Class 3.

Clip	Topic	Min
1	Why bits go wrong	8
2	Framing: finding the frame	13
3	The idea, and parity bits	12
4	2D parity and the card trick	12
5	The Internet checksum	15
6	CRC: computing it	16
7	Checking CRC, and wrap-up	14

CLIP 1 OF 3

Why bits go wrong



IN THIS CLIP

- A quick recap of Class 1
- Where errors come from
- Single-bit errors and burst errors

Where we left off in Class 1

Three ideas to carry into today

A frame crosses one link

Each hop is a separate truck trip with its own trip sheet.

IP stays, MAC changes

IP is end to end. MAC covers one hop only.

ARP maps IP to MAC

Inside one subnet. For other subnets, ARP for the gateway.

THE ETHERNET FRAME FROM CLASS 1

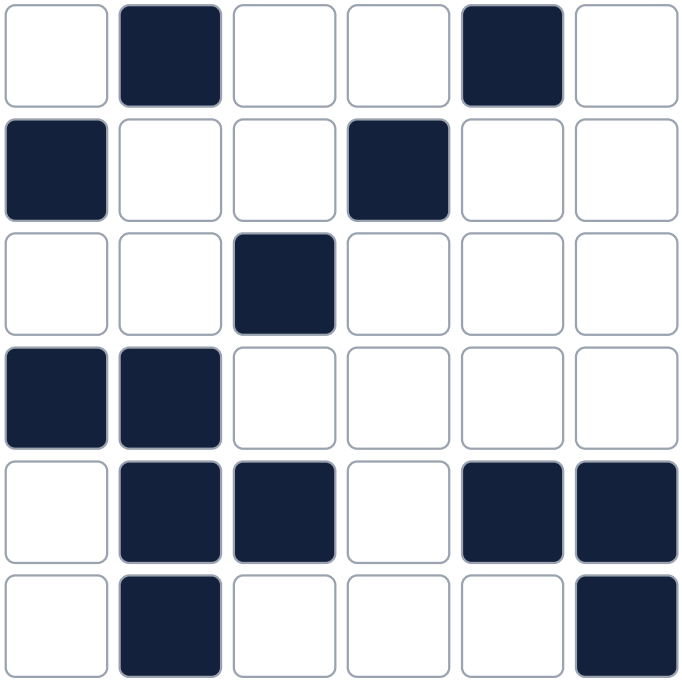


Framing: where does the frame start?
Clip 1

Error detection: did a bit change?
Clips 3

Remember this grid?

From the end of Class 1



Someone flipped one card.

Did you find it? By the end of Clip 3 you will find it in seconds, and you will know exactly why the trick works.

The same idea protects real data on real links.

Why do bits go wrong?

Layer 1 problems that Layer 2 has to catch



Noise

Random electrical energy adds to the signal, so a 0 can look like a 1.



Interference

Other signals on the same frequency. A microwave oven can disturb 2.4 GHz Wi-Fi.



Attenuation

The signal gets weaker with distance until it is hard to read.



Crosstalk

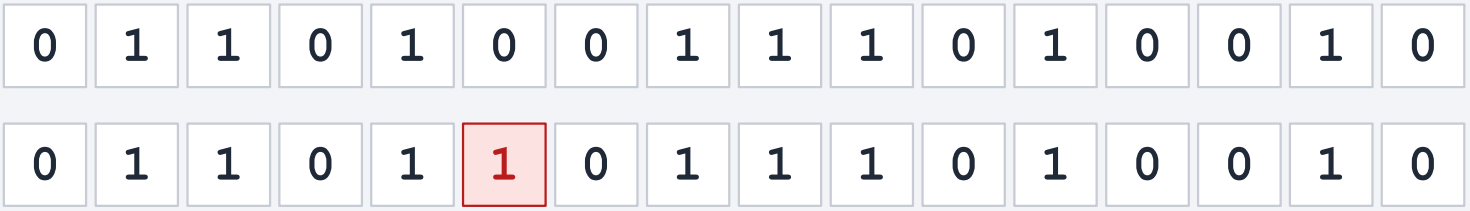
Wires close together leak signals into each other.

Parcel story: the road damages the box (Layer 1). Staff at the next hub inspect it (Layer 2). Details of the road come in Classes 4 and 5.

Two kinds of damage

Single-bit errors and burst errors

SINGLE-BIT ERROR



BURST ERROR (LENGTH 6)



By the end of this video, you can…

Six learning outcomes for Class 2

1. Explain how a receiver finds the start and end of a frame
2. Apply byte stuffing and bit stuffing, in both directions
3. Compute and check parity, including 2D parity
4. Compute and verify an Internet checksum
5. Compute and verify a CRC
6. Compare the methods and say what happens after an error

Framing: finding the frame



IN THIS CLIP

- Why a bit stream has no boundaries
- Byte count, and why it is fragile
- Flags with byte stuffing and bit stuffing
- What Ethernet actually does

Layer 1 delivers bits, not frames

Before checking a frame, the receiver has to find it

WHAT ARRIVES AT THE NETWORK CARD

0 1 1 0 1 0 0 1 1 1 0 0 1 0 1 1 0 1 0 0 1 1 1 0 0 1 0 1 ...

No gaps. No labels. No pause between frames.

Boundary found correctly

Every header field lands where it should. The destination MAC, the type field and the error check are all read from the right bits.

Boundary off by a few bits

Every field is read from the wrong bits. The frame looks damaged even if no bit changed, and the next frame is wrong too.

Framing is adding markers so the boundary can be found. Like putting items on a conveyor belt into sealed boxes. There are three classic ways to do it.

Idea 1: a byte count

The first byte says how many bytes follow

WHEN NOTHING IS DAMAGED



ONE BIT FLIPS IN THE COUNT: 5 BECOMES 7



Everything after the boundary is misread

Header fields, addresses and the error check are all taken from the wrong bytes.

The receiver never recovers

The next length field is really a data byte, so every following frame is wrong too.

This is why no real protocol uses a byte count on its own.

Idea 2: a flag byte, with byte stuffing

PPP [used by DSL] marks both ends with the flag 0x7E = 0111 1110



The two rules

```
Data byte 7E  →  7D 5E      (7E XOR 20 = 5E)
Data byte 7D  →  7D 5D      (7D XOR 20 = 5D)
```

The receiver reverses it

See 7D? Delete it and XOR the next byte with 0x20. Any 7E left is a real flag.

II PAUSE THE VIDEO · TRY ON PAPER

Try it: byte stuffing



Q1 The payload is 7E 41 7D. What does the sender transmit between the flags?

Q2 The receiver gets 42 7D 5E 43 between the flags. What was the original payload?

Hint: a data 7E becomes 7D 5E, and a data 7D becomes 7D 5D. The receiver does the opposite.

ANSWERS

Byte stuffing answers

Q1 · sender

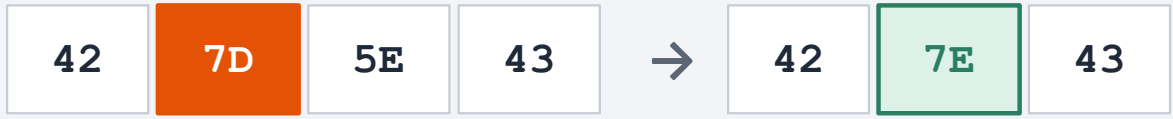
payload



3 bytes become 5

Q2 · receiver

received



5E XOR 20 = 7E

Common mistake: keeping the escape byte 7D in the de-stuffed payload.

Idea 3: the same trick, one bit at a time

HDLC [used by Cisco router] uses the flag pattern 0111 1110 — a 0, six 1s, a 0

THE FLAG

0	1	1	1	1	1	1	0

six 1s in a row: this pattern must never appear inside the data

DATA THE SENDER WANTS TO SEND (IT LOOKS LIKE THE FLAG!)

0	1	1	1	1	1	1	0

five 1s in a row

WHAT ACTUALLY GOES ON THE WIRE

0	1	1	1	1	1	0	1	0

↑ a 0 inserted by the sender

Sender rule

After five 1s in a row, insert a 0 — whatever comes next. Overhead is at most one bit in six.

Receiver rule

After five 1s: a 0 next means delete it (the sender put it there). A 1 next means this is the flag — the frame has ended.

What Ethernet actually does

A preamble to lock on, a delimiter to start, silence to end



Preamble: 7 bytes of 1010…

A steady, clock-like signal. The receiver uses it to lock on to the sender's bit timing (Class 4).

SFD: 1010 1011

The last two bits are two 1s, which never happen in the preamble. That says: the real frame starts now.

The end: silence

The sender stops transmitting and a mandatory quiet gap follows, so no end marker is needed in the data.

No stuffing is needed: the marker lives in the signal, not in the data bits.

Four ways to mark a frame

And one rule about the order of operations

Method	Marker	Cost	Weakness	Used by
Byte count	Length field at the start	None	One damaged count breaks the stream	Rare on its own
Flag + byte stuffing	Flag byte 0x7E	A few extra bytes	Payload grows (worst case x2)	PPP
Flag + bit stuffing	Flag 0111 1110	At most 1 bit in 6	Bit-level, suits hardware	HDLC
Preamble + silence	Preamble and SFD	None in the data	Needs physical-layer support	Ethernet, Wi-Fi



Framing first, error detection second.
The receiver must find the frame boundaries before it can check the frame. If framing fails, the error check is computed over the wrong bits and is meaningless.

Clip 1 in three lines

1. Layer 1 delivers a stream of bits, so Layer 2 has to mark where each frame begins and ends.
2. A flag byte or flag pattern needs stuffing, because the flag could otherwise appear inside the data.
3. Ethernet uses a preamble and a start frame delimiter, and silence marks the end.

Next clip: now that we can find a frame, did it arrive undamaged?

CLIP 2 OF 3

The idea, and parity bits

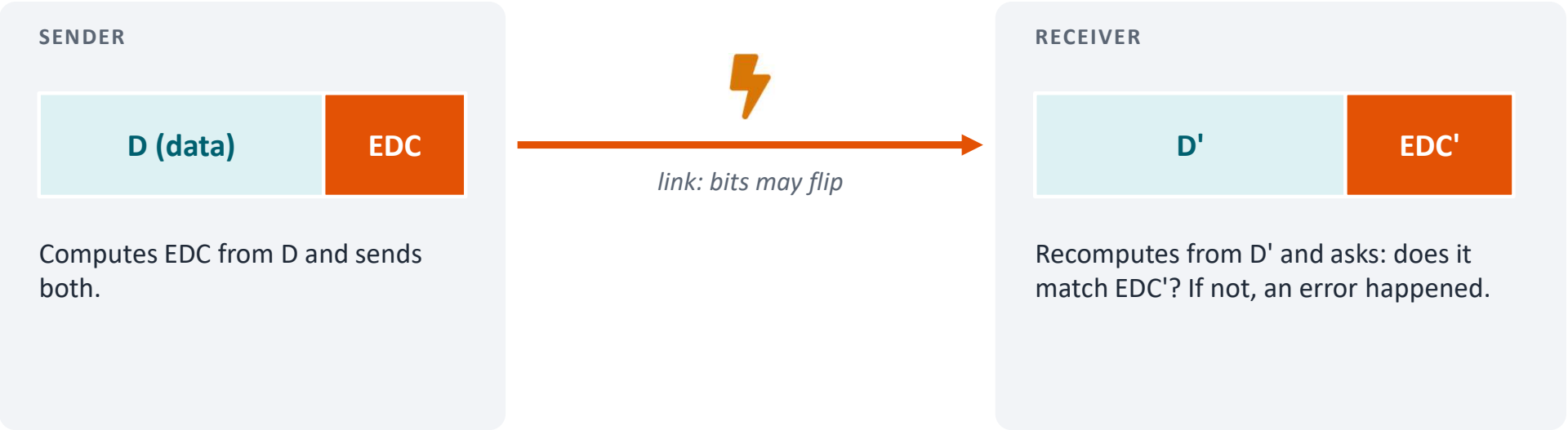


IN THIS CLIP

- Adding redundant bits
- Why no method is perfect
- Single parity bit: what it catches and misses

The basic idea: send a little extra

Error-detection bits (EDC) are computed from the data



Computes EDC from D and sends both.

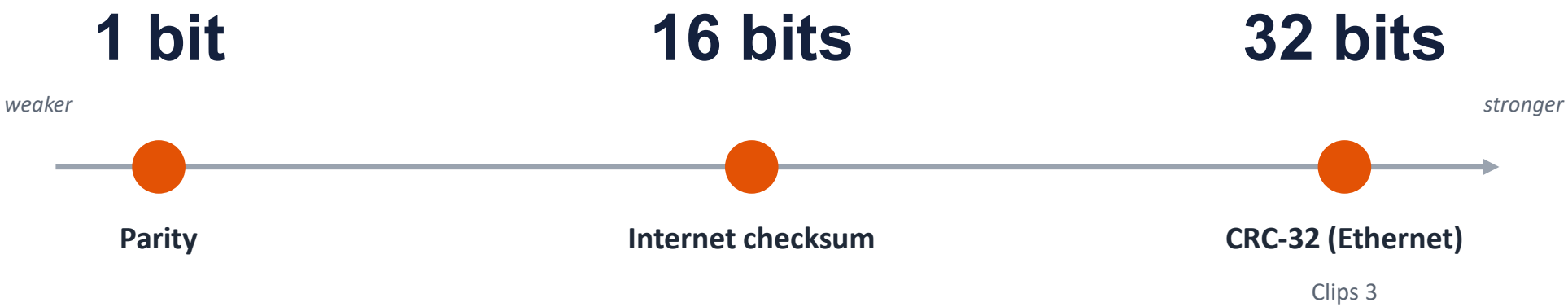
Recomputes from D' and asks: does it match EDC'? If not, an error happened.

The ' means "what actually arrived". It may differ from what was sent, and the receiver cannot see the original.

Parcel story: a packing list says "3 items, 2.5 kg". The hub weighs the box and compares.

No method is perfect

More extra bits catch more errors, but cost more



Clips 3

Undetected errors can still happen
If the damage happens to produce a matching EDC, the receiver is fooled. Good methods make this very rare.

Every extra bit is overhead
It carries no user data. Designers balance protection against cost and speed.

The simplest method: one parity bit

Even parity: make the total number of 1s even

DATA (8 BITS)

1	0	1	1	0	1	1	0

Count the 1s: 1 + 1 + 1 + 1 + 1 = 5 (odd)

So the parity bit is 1, making the total 6 (even).

SENT (9 BITS)

1	0	1	1	0	1	1	0	1

The rule

- Odd number of 1s → parity bit 1
- Even number of 1s → parity bit 0
- Receiver counts all 9 bits: even = OK, odd = error

Odd parity is the mirror image: make the total odd.

Parity catches one flipped bit…

Bit 3 flips on the way

SENT

1	0	1	1	0	1	1	0	1

RECEIVED

1	0	0	1	0	1	1	0	1

Error detected

The receiver counts five 1s. Five is odd, but even parity promised an even count.

Note: the receiver knows something is wrong, but not which bit. It cannot fix the error.

…but misses two

Bits 3 and 4 both flip

SENT



RECEIVED



Looks fine (wrong!)

Four 1s is even, so the receiver accepts damaged data.

Rule: one parity bit catches any odd number of flipped bits and misses any even number. Burst errors often flip several bits, so this is weak.

II PAUSE THE VIDEO · TRY ON PAPER

Try it: even parity

Q1 What is the even parity bit for the data 0110 1101?

Q2 The receiver gets 1100 1010 1 (8 data bits + parity). Is there an error?

Q3 The receiver gets 0101 0101 0. Does it accept the data? Can it be sure the data is correct?

Hint: count every 1, including the parity bit when you are the receiver.

ANSWERS

Even parity answers

Q1 0 1 1 0 1 1 0 1 **1**

Five 1s is odd, so the parity bit is 1.

Q2 1 1 0 0 1 0 1 0 **1**

Five 1s in total is odd: error detected.

Q3 0 1 0 1 0 1 0 1 **0**

Four 1s is even: accepted. But it cannot be sure; two flipped bits would look the same.

Clip 3 in three lines

1. The sender adds error-detection bits computed from the data. The receiver recomputes and compares.

2. No method is perfect. More bits give better protection but more overhead.

3. One parity bit catches an odd number of flipped bits and misses an even number. It can detect, but not fix.

Next clip: what if we add parity in two directions?

2D parity and the card trick



IN THIS CLIP

- Parity on rows and columns
- Finding and fixing one flipped bit
- The card trick revealed
- Detect-and-resend vs fix-it-yourself

Two-dimensional parity

Arrange the data in a grid, then add parity to every row and every column

DATA: 3 ROWS × 5 BITS

1	1	0	1	0
0	1	1	0	0
1	0	1	1	1

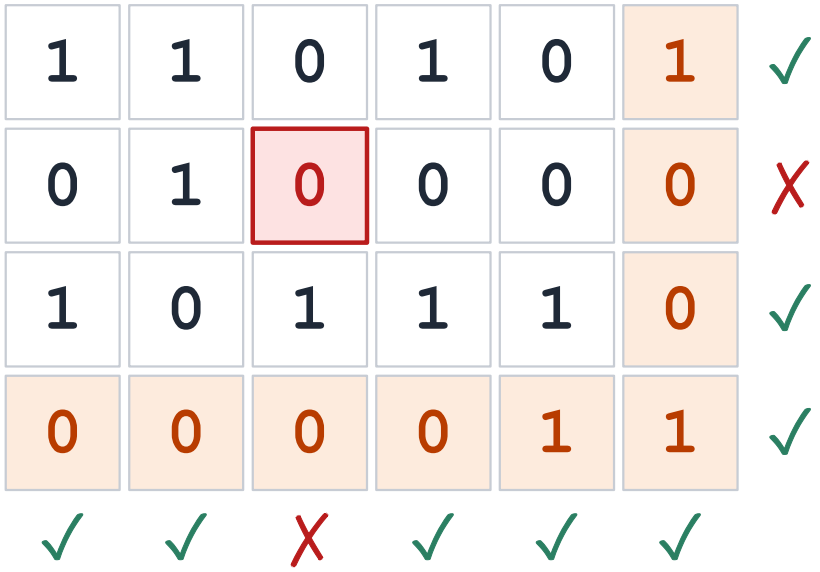
WITH PARITY ADDED

1	1	0	1	0	1	← row parity
0	1	1	0	0	0	
1	0	1	1	1	0	
0	0	0	0	1	1	← column parity

Every row and every column now has an even number of 1s, including the parity row and the parity column. The corner bit checks the parity bits themselves.

One flipped bit: find it and fix it

The bad row and the bad column cross at the error



1. Check every row: row 2 is odd ✗
2. Check every column: column 3 is odd ✗
3. **The error is where they cross: row 2, column 3**
4. **Flip that bit back from 0 to 1. Fixed!**

This is error correction: no need to ask the sender again.

II PAUSE THE VIDEO · TRY ON PAPER

Try it: find the flipped bit



1	1	0	1	0	1
0	1	1	0	0	0
0	0	1	1	1	0
0	0	0	0	1	1

One bit was flipped on the way.

Which row and column is it in, and what should its value be?

Hint: check the parity of every row and every column, including the orange ones.

ANSWER

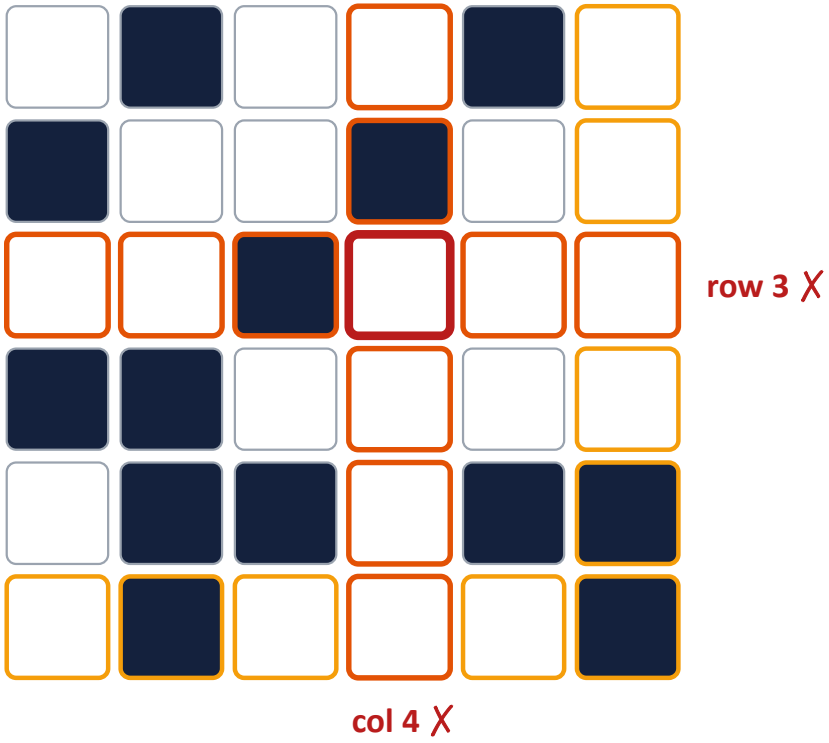
Row 3, column 1: it should be 1

1	1	0	1	0	1	✓
0	1	1	0	0	0	✓
0	0	1	1	1	0	✗
0	0	0	0	1	1	✓
✗	✓	✓	✓	✓	✓	

- Row 3 has three 1s: odd ✗
- Column 1 has one 1: odd ✗
- All other rows and columns are even ✓
- **So the bit at row 3, column 1 flipped from 1 to 0. Flip it back to 1.**

The card trick, revealed

The Class 1 grid was 2D parity all along



1. The top-left 5 × 5 cards are the "data".
2. I secretly added the last row and last column so every row and column had an even number of white cards.
3. Flipping one card makes exactly one row and one column odd.
4. Row 3 and column 4 cross at the flipped card.

White = 1, dark = 0. Orange borders: the hidden parity row and column.

Limits of 2D parity

Two flipped bits in the same row

1	1	0	1	0	1	✓
0	0	1	0	1	0	✓
1	0	1	1	1	0	✓
0	0	0	0	1	1	✓
✓	✗	✓	✓	✗	✓	

- Row 2 still has an even count ✓
- Columns 2 and 5 are odd ✗ ✗
- **The receiver knows something is wrong: detected.**
- **But no row points to the error, so it cannot tell which bits to fix: not correctable.**

Summary: 2D parity corrects any single-bit error and detects any 2- or 3-bit error. Some 4-bit patterns can slip through.

After spotting an error: resend or fix?

Two strategies



Detect and resend

Ask for the box again

- Receiver only needs to detect errors
- Sender retransmits the damaged frame
- Example: Wi-Fi resends frames that were not acknowledged

Good when resending is quick and cheap.



Forward error correction (FEC)

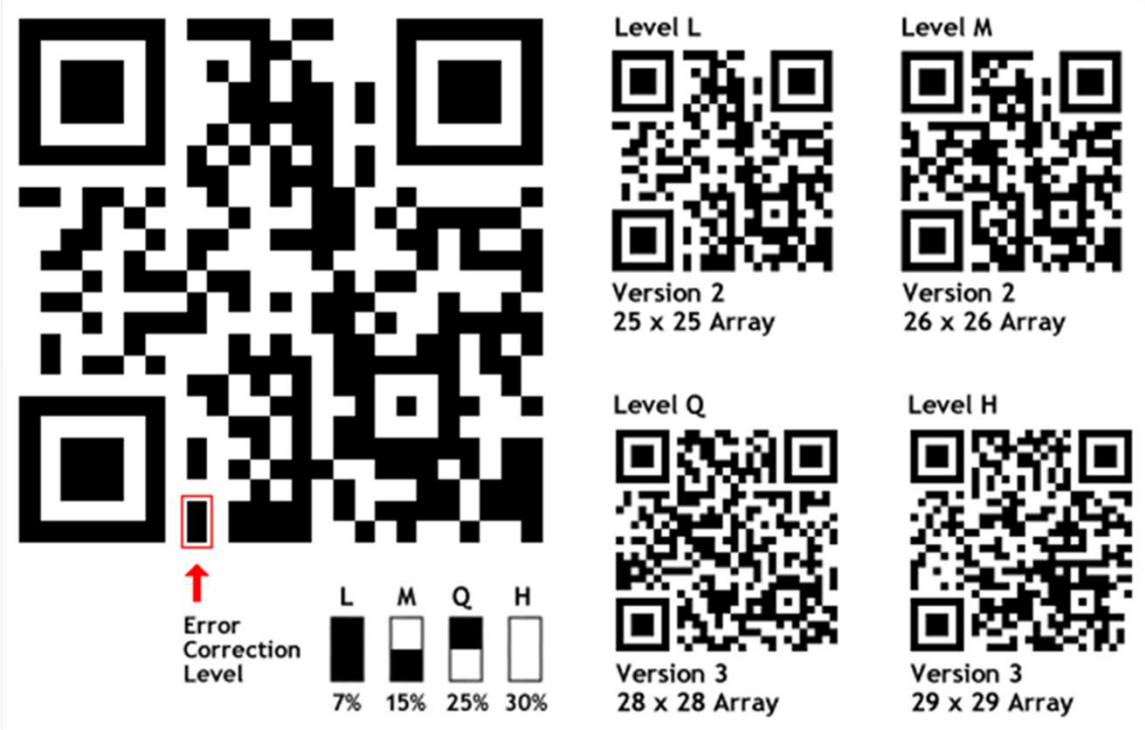
Fix the box yourself

- Send enough extra bits to repair errors
- No waiting for a resend
- Examples: QR codes, 5G and Wi-Fi coding, deep-space probes

Good when resending is slow or impossible.

QR code FEC

Based on Reed-Solomon error correction algorithm



Source: https://support.bradyid.com/servlet/rtaImage?eid=ka04V000001HqoK&feoid=00N3A00000DGCTR&refid=0EM3A0000023HWV

Clip 4 in three lines

1. 2D parity adds a parity bit to every row and every column.
2. One flipped bit makes one row and one column odd, so we can find it and fix it.
3. Links either detect and resend, or send extra bits to fix errors on arrival (FEC).

Next clip: a method you have already seen in TCP and UDP headers.

The Internet checksum



IN THIS CLIP

- Adding data words together
- One's complement addition with wraparound
- A good frame and a damaged one
- Weaknesses, and where checksums are used

Checksum: add everything up

Used in the IPv4, UDP, and TCP headers you studied before



1. Split

Cut the data into fixed-size words (16 bits on the Internet).



2. Add

Add all the words using one's complement addition.



3. Flip

Invert every bit of the sum. That is the checksum.



4. Check

Receiver adds all words plus the checksum. All 1s means OK.

To keep examples short, we use 8-bit words. The steps are identical for 16-bit words.

Parcel story: the packing list says the total weight. If the box weighs something else, something changed.

One's complement addition: the wraparound

The only new rule is what to do with a carry out of the left end

EXAMPLE WITH A CARRY

```
          1011 1011
+         1000 1111
=       1 0100 1010
wrap      0100 1010
+                 1
=         0100 1011
```

1. Add the two words in normal binary.
2. If a 1 is carried out of the leftmost bit, remove it…
3. …and add it back to the rightmost bit (wraparound).
4. At the very end, flip all bits to get the checksum.

Decimal check: 187 + 143 = 330. Remove 256, add 1: 75 = 0100 1011.

Worked example: the sender

Three 8-bit data words

Step	Binary	Decimal	What happened
Word 1	0110 0110	102	
Word 2	0101 0101	85	
Word 1 + Word 2	1011 1011	187	No carry out
+ Word 3	1000 1111	143	
Raw sum	1 0100 1010	330	Carry out of the left end!
After wraparound	0100 1011	75	Drop the carry, add 1
Checksum (flip all bits)	1011 0100	180	Sent with the data

Worked example: the receiver, frame intact

Add all the words and the checksum

Sum of 3 words	0100 1011
+ checksum	1011 0100
=	1111 1111

All 1s → no error detected
A number plus its bit-flipped copy is always all 1s.

Any 0 → error detected
Something changed on the way; the data is discarded.

In decimal: 75 + 180 = 255, which is 1111 1111.

Same example, but one bit is damaged

Word 3 was sent as 1000 1111 and arrived as 1000 1011

WORD 3 AS SENT

1	0	0	0	1	1	1	1

WORD 3 AS RECEIVED

1	0	0	0	1	0	1	1

The receiver has no idea: it just adds what it has.

Word 1 + Word 2	1011 1011	187
+ damaged word 3	1000 1011	139
raw sum	1 0100 0110	carry
after wraparound	0100 0111	71
+ checksum	1011 0100	180
=	1111 1011	251

Not all 1s — there is a 0 in position 5. Error detected, frame discarded.
Pause and check this arithmetic yourself. It is exactly what an exam question looks like.

II PAUSE THE VIDEO · TRY ON PAPER

Try it: two words

Q1 Compute the 8-bit checksum of 1100 1010 and 0011 0110.

Q2 Check your answer the way the receiver would. Do you get all 1s?

Hint: watch for a carry out of the leftmost bit.

ANSWER

Checksum = 1111 1110

	1100 1010	202
+	0011 0110	54
=	1 0000 0000	256: carry!
wrap	0000 0001	1
flip	1111 1110	checksum

Receiver check (Q2)

```
  0000 0001
+ 1111 1110
= 1111 1111
```

All 1s: no error detected. ✓

II PAUSE THE VIDEO · TRY ON PAPER

Try it again: three words

Q1 Compute the 8-bit checksum of 1001 0110, 0110 1101 and 1100 0011.

Q2 Verify it as the receiver: add all three words and the checksum.

Hint: add two words, wrap if needed, then add the third. Do not wait until the end to wrap.

ANSWER

Checksum = 0011 1000

Step	Binary	Decimal	What happened
Word 1 + Word 2	1 0000 0011	259	Carry out — wrap now, not later
After wraparound	0000 0100	4	Drop the carry, add 1
+ Word 3	1100 0011	195	
Running total	1100 0111	199	No carry this time
Checksum (flip all bits)	0011 1000	56	Sent with the data
Receiver: 199 + 56	1111 1111	255	All 1s — no error detected

Common mistake: adding all three words first and only wrapping at the end. With 8-bit words the sum can overflow twice, so wrap after each addition.

Where the checksum falls short

And why the link layer uses something stronger



Order does not matter

Swap two words and the sum is the same, so the error is missed.



Errors can cancel

If one word goes up by the same amount another goes down, the sum does not change.

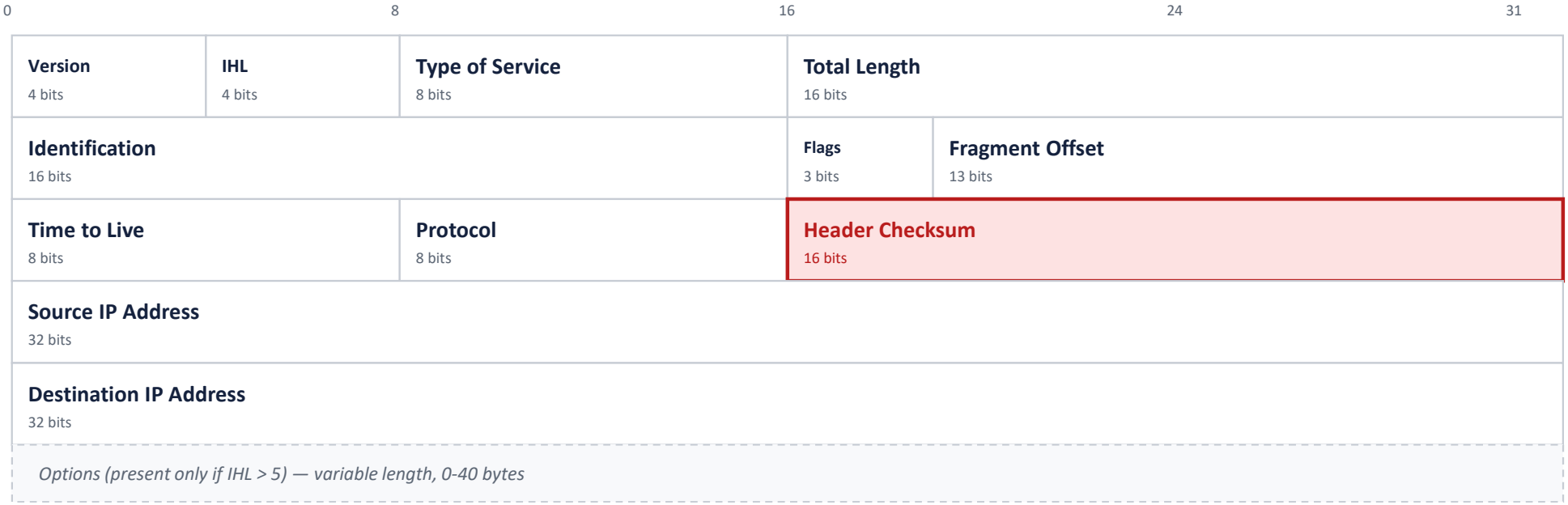
So why use it?

- Very simple and fast in software
- Used by IPv4, UDP, and TCP (Layers 3-4)
- Only a last line of defense: the link layer already checked each frame

Link layer: network cards compute a CRC in hardware. It is much stronger, and hardware makes it fast. That is the next clip.

IPv4 Header structure

The 16-bit Header Checksum works the same way as the Layer 4 checksum, one layer down



Recap: the Header Checksum covers only the 20-byte header, is 16 bits wide, and uses the same ones'-complement sum as the Layer 4 (TCP/UDP) checksum — it verifies integrity, it does not correct errors.

Clip 5 in three lines

1. Add the data words with one's complement addition: wrap any carry around to the right.

2. The checksum is the sum with every bit flipped. The receiver expects all 1s.

3. Simple and fast in software, but it misses reordered words and errors that cancel out.

Next clip: the CRC, the field at the end of every Ethernet frame.

CLIP 3 OF 3

CRC: computing it



IN THIS CLIP

- The big idea behind CRC
- Modulo-2 arithmetic (XOR)
- The four-step recipe
- Two full worked examples, then your turn

Cyclic redundancy check (CRC): the big idea

Choose the extra bits so the whole frame divides evenly



G	The generator: r + 1 bits, agreed in advance by sender and receiver (it is part of the standard).
Sender	Picks R so that the bits D followed by R divide by G with no remainder.
Receiver	Divides what arrived by G. Remainder 0: accept. Anything else: error.

Similar idea: the last digit of a Thai national ID number is computed from the first 12 so typing mistakes can be caught.

Modulo-2 arithmetic is just XOR

No carries, no borrows. Adding and subtracting are the same thing.

a	b	a XOR b
0	0	0
0	1	1
1	0	1
1	1	0

1 XOR 1 = 0, with no carry

EXAMPLES

```
  1011          1001
XOR 0101      XOR 1101
------        ------
  1110          0100
```

Compare bit by bit: same → 0, different → 1.

Why XOR? It needs only one tiny logic gate per bit, which is why network cards can compute CRCs at very high speed. We see the circuit in Clip 7.

The recipe: computing R

Long division, but with XOR instead of subtraction

1	**Append zeros**	Write D and add r zeros at the end (r = number of bits in G minus 1).
2	**Look at the leading bit**	Take the first r + 1 bits. If the leading bit is 1, XOR with G. If it is 0, XOR with zeros.
3	**Drop and bring down**	Drop the leading bit (now 0) and bring down the next bit. Repeat until no bits are left to bring down.
4	**Read R**	The last r bits are the remainder R. Send D followed by R.

You always do exactly d steps: one for each data bit.

Example 1: setting up

D = 110101 (d = 6), G = 1011 (4 bits, so r = 3)



STEP 1: APPEND r = 3 ZEROS



Now divide 110101000 by 1011 using XOR. Six data bits means six steps. The next slide shows every step.

Tip: pause here and try it yourself first.

Example 1: the division

Dividend 110101000, generator 1011

1	1	0	1	0	1	0	0	0

bring-down bits come from here, left to right

Step	Current bits	Leading bit	XOR with	Result	Next: drop 1st bit, bring down 1
1	1101	1	1011	0110	1100
2	1100	1	1011	0111	1111
3	1111	1	1011	0100	1000
4	1000	1	1011	0011	0110
5	0110	0	0000	0110	1100
6	1100	1	1011	0111	**R = 111**

Step 5 starts with 0, so we XOR with 0000 (nothing changes) and just move on.

Example 1: what gets sent

Replace the three zeros with R = 111

FRAME BITS SENT



Promise: 110101111 divided by 1011 leaves remainder 000. If the receiver gets anything else, it knows the frame was damaged. We check this in Clip 7.

In Ethernet, R is the 4-byte CRC field (r = 32) at the end of the frame.

Example 2: a different generator

D = 100100 (d = 6), G = 1101 (4 bits, so r = 3)



DIVIDEND AFTER APPENDING 3 ZEROS



Watch for this

One of the steps starts with a 0 again, so you XOR with 0000 and simply move on.

And this

The answer starts with a zero. R always has exactly r bits — leading zeros count.

Pause here and do the whole division yourself. The full answer is on the next slide.

Example 2: the division

Dividend 100100000, generator 1101

1 0 0 1 0 0 0 0 0 *bring-down bits come from here, left to right*

Step	Current bits	Leading bit	XOR with	Result	Next: drop 1st bit, bring down 1
1	1001	1	1101	0100	1000
2	1000	1	1101	0101	1010
3	1010	1	1101	0111	1110
4	1110	1	1101	0011	0110
5	0110	0	0000	0110	1100
6	1100	1	1101	0001	**R = 001**

FRAME BITS SENT

1 0 0 1 0 0 0 0 1

R is 001, not 1 — dropping the leading zero is the most common exam mistake.

II PAUSE THE VIDEO · TRY ON PAPER

Now one completely on your own

Q1 D = 11010 and G = 101. How many zeros do you append, and how many steps will you do?

Q2 Compute the remainder R.

Q3 Write the full bit string the sender transmits.

Hint: r = 2 here. When the leading bit is 0, XOR with 000.

ANSWER

R = 01, so the sender transmits 1101001

Q1: append 2 zeros (dividend 1101000); 5 data bits means 5 steps.

Step	Current bits	Leading bit	XOR with	Result	Next: drop 1st bit, bring down 1
1	110	1	101	011	111
2	111	1	101	010	100
3	100	1	101	001	010
4	010	0	000	010	100
5	100	1	101	001	**R = 01**

1 1 0 1 0 **0 1** ← Q3: D followed by R

Clip 6 in three lines

1. Sender and receiver agree on a generator G with r + 1 bits.
2. The sender appends r zeros, divides by G using XOR, and keeps the r-bit remainder R.
3. It sends D followed by R, which is guaranteed to divide evenly by G.

Next clip: how the receiver checks, how hardware does it, and why CRC is so strong.

Checking CRC, and wrap-up



IN THIS CLIP

- The receiver side of CRC
- How a network card does it in hardware
- Standard generators and why CRC is strong
- Comparing all four methods, and your two assignments

The receiver: divide and look at the remainder

Same division, no zeros appended this time

Frame arrives intact

RECEIVED

1	1	0	1	0	1	1	1	1

÷ 1011 → remainder **000**

Remainder 000: accept the frame

One bit flipped on the way

RECEIVED

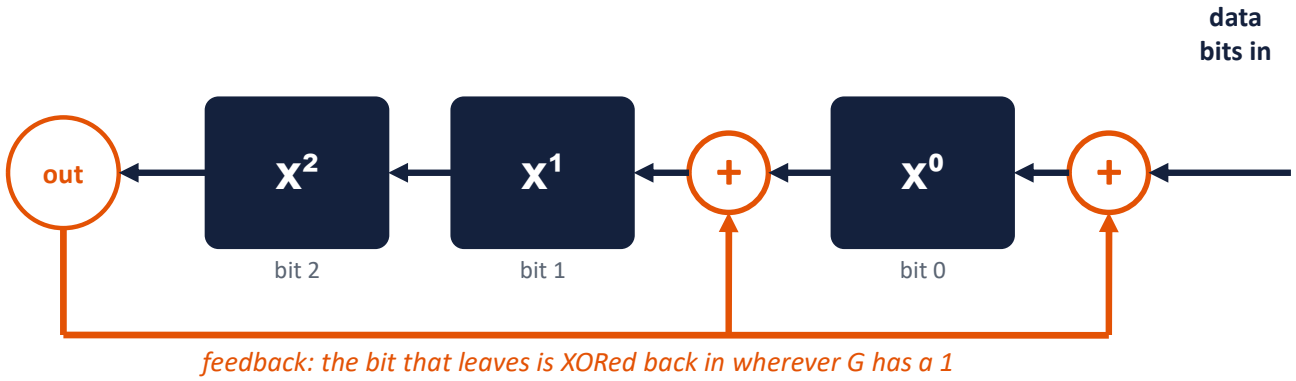
1	1	0	1	1	1	1	1	1

÷ 1011 → remainder **110**

Remainder 110: error detected, drop the frame

How a network card really does it

Not long division on paper: a shift register with XOR gates



Where the gates go

`G = 1011`

x^3	1	the bit that leaves
x^2	0	no gate
x^1	1	gate
x^0	1	gate

A 3-bit register for G = 1011. After the last bit, the register holds R. CRC-32 uses 32 cells — the same picture, only wider.

One bit per clock tick

The register shifts once for each arriving bit. No long division, no loops.

A handful of gates

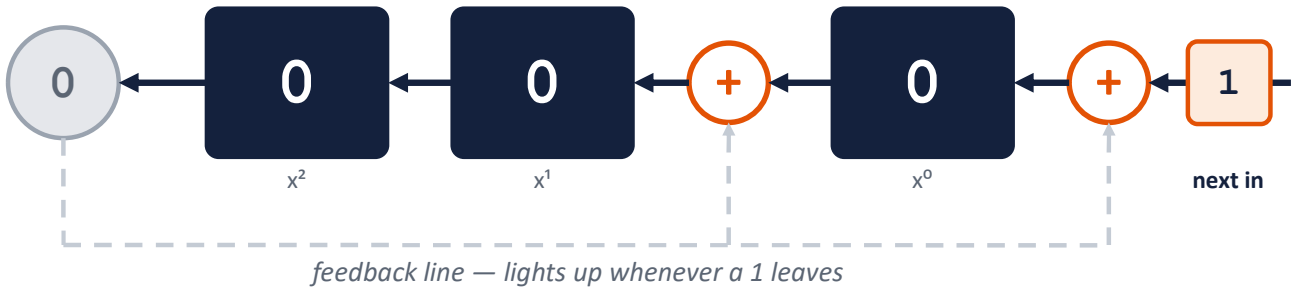
A few flip-flops and XOR gates. Cheap enough to sit in every network card.

No buffering, no delay

It is computed while the bits fly past, so it keeps up at 10 Gbps and beyond.

Watch it run: Example 1 in the circuit

9 clock ticks, one bit per tick — 110101 plus three 0s, exactly the long division from Clip 6



the register starts empty: 000

first bit waiting to enter →

1 The leftmost bit leaves

The bit in x^2 falls out on the left. It is the feedback bit — the leading bit of the long division window.

2 Everything shifts one place left

The next data bit enters on the right, into x^0. Nothing is buffered; the bits flow straight through.

3 If a 1 left, the gates flip

Each gate XORs its bit with the feedback. Feedback 1 flips the bit; feedback 0 leaves it. That is “XOR with 1011”.

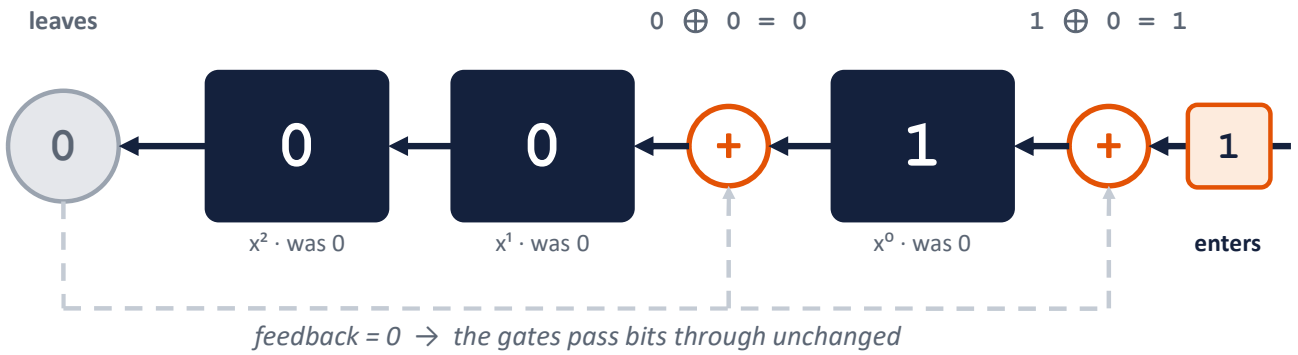
All three happen at once, on every tick. Orange cells were flipped by a gate.

EXAMPLE 1 IN THE CIRCUIT · TICK 1 OF 9

Filling the register

stream: 1 1 0 1 0 1 0 0 0

teal = data 110101 · orange = the three appended 0s



No division yet

Only 0s have left the register, so the gates do nothing. Bits simply shift in — like copying the first digits before a long division can start.

The first window, 1101, is complete at tick 4.

REGISTER AFTER EACH TICK

tick 1	tick 2	tick 3	tick 4	tick 5	tick 6	tick 7	tick 8	tick 9
001								

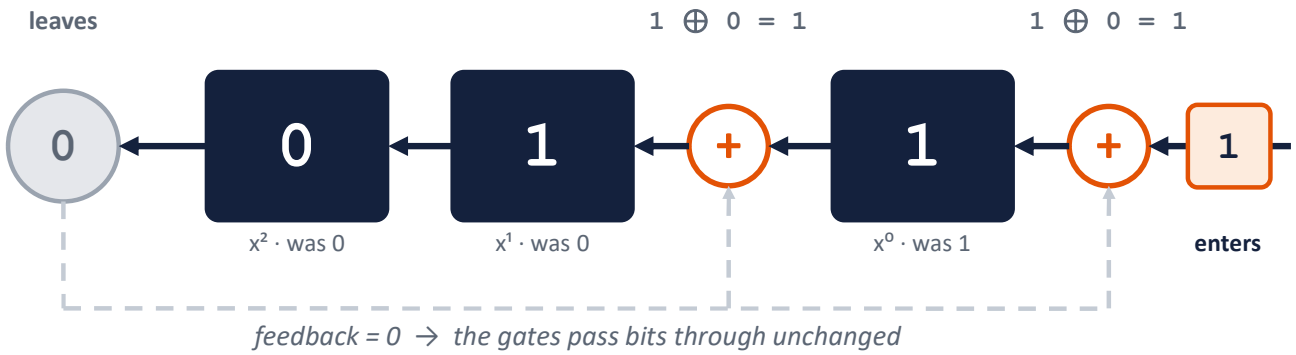
The first bit, 1, enters on the right. A 0 leaves on the left, so the gates do nothing.

EXAMPLE 1 IN THE CIRCUIT · TICK 2 OF 9

Filling the register

stream: 1 **1** 0 1 0 1 0 0 0

teal = data 110101 · orange = the three appended 0s



No division yet

Only 0s have left the register, so the gates do nothing. Bits simply shift in — like copying the first digits before a long division can start.

The first window, 1101, is complete at tick 4.

REGISTER AFTER EACH TICK

tick 1	tick 2	tick 3	tick 4	tick 5	tick 6	tick 7	tick 8	tick 9
001	011							

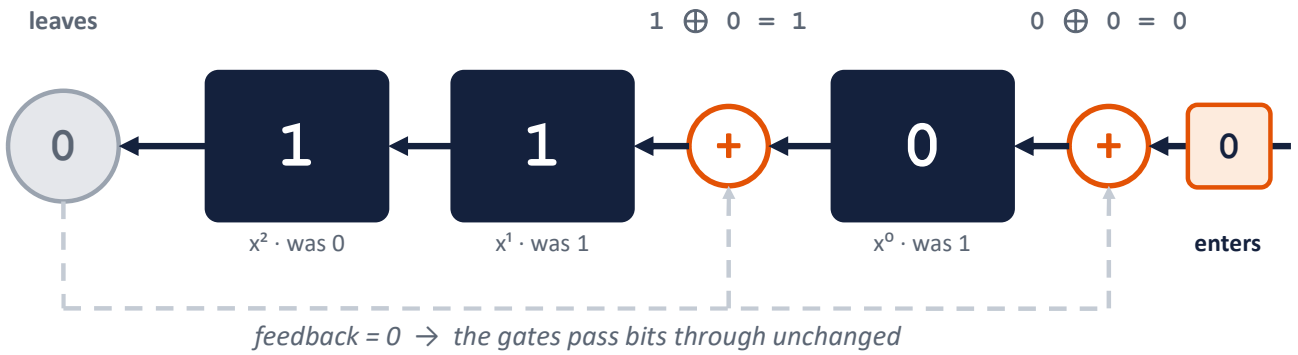
The second bit, 1, enters. Still only zeros leave — the register is filling up.

EXAMPLE 1 IN THE CIRCUIT · TICK 3 OF 9

Filling the register

stream: 1 1 0 1 0 1 0 0 0

teal = data 110101 · orange = the three appended 0s



No division yet

Only 0s have left the register, so the gates do nothing. Bits simply shift in — like copying the first digits before a long division can start.

The first window, 1101, is complete at tick 4.

REGISTER AFTER EACH TICK

tick 1	tick 2	tick 3	tick 4	tick 5	tick 6	tick 7	tick 8	tick 9
001	011	110						

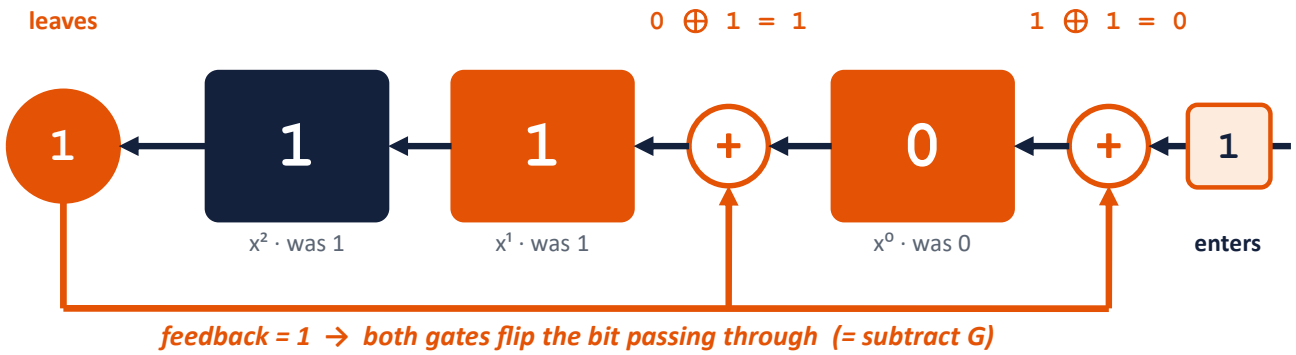
The third bit, 0, enters. The register now holds 110 — the start of the first window.

EXAMPLE 1 IN THE CIRCUIT · TICK 4 OF 9

Tick 4 = long-division step 1

stream: 1 1 0 **1** 0 1 0 0 0

teal = data 110101 · orange = the three appended 0s



Same as step 1 of the table

window	1101
leading bit	1
XOR with	1011
result	0110

last 3 bits 110 = **register ✓**

REGISTER AFTER EACH TICK

tick 1	tick 2	tick 3	tick 4	tick 5	tick 6	tick 7	tick 8	tick 9
001	011	110	110					

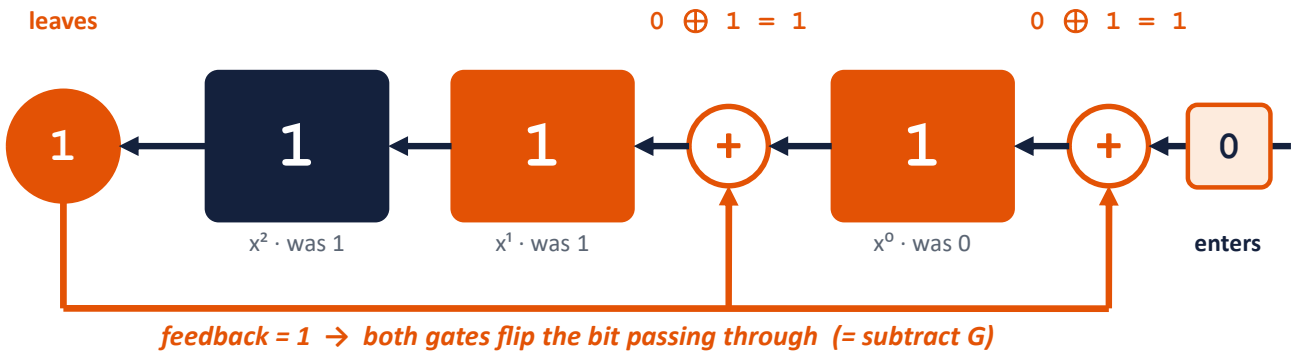
A 1 leaves: that is the leading 1 of window 1101. Feedback 1 means "subtract G", so the gates flip x^1 and x^0.

EXAMPLE 1 IN THE CIRCUIT · TICK 5 OF 9

Tick 5 = long-division step 2

stream: 1 1 0 1 **0** **1** **0** **0** **0**

teal = data 110101 · orange = the three appended 0s



Same as step 2 of the table

window	1100
leading bit	1
XOR with	1011
result	0111

last 3 bits 111 = **register ✓**

REGISTER AFTER EACH TICK

tick 1	tick 2	tick 3	tick 4	**tick 5**	tick 6	tick 7	tick 8	tick 9
001	011	110	110	**111**				

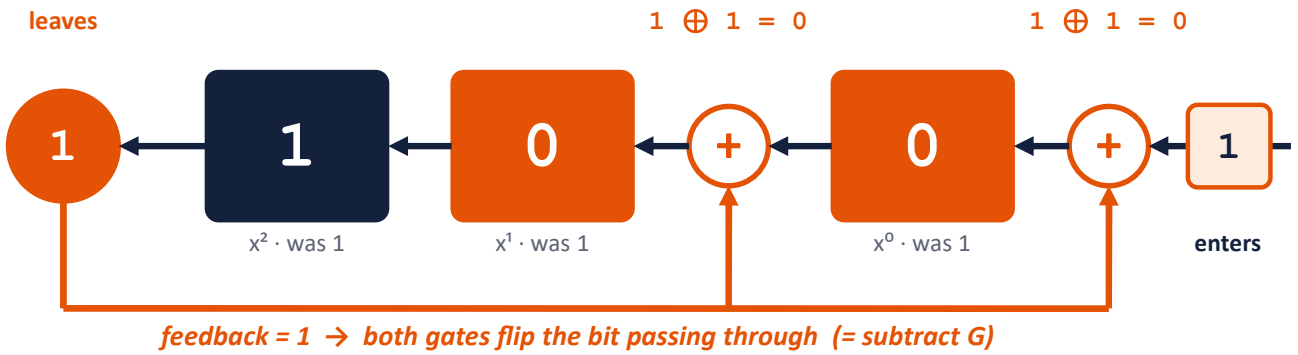
Another 1 leaves. The gates flip again: window 1100 XOR 1011 = 0111.

EXAMPLE 1 IN THE CIRCUIT · TICK 6 OF 9

Tick 6 = long-division step 3

stream: 1 1 0 1 0 **1** 0 0 0

teal = data 110101 · orange = the three appended 0s



Same as step 3 of the table

window	1111
leading bit	1
XOR with	1011
result	0100

last 3 bits 100 = **register** ✓

REGISTER AFTER EACH TICK

tick 1	tick 2	tick 3	tick 4	tick 5	**tick 6**	tick 7	tick 8	tick 9
001	011	110	110	111	**100**			

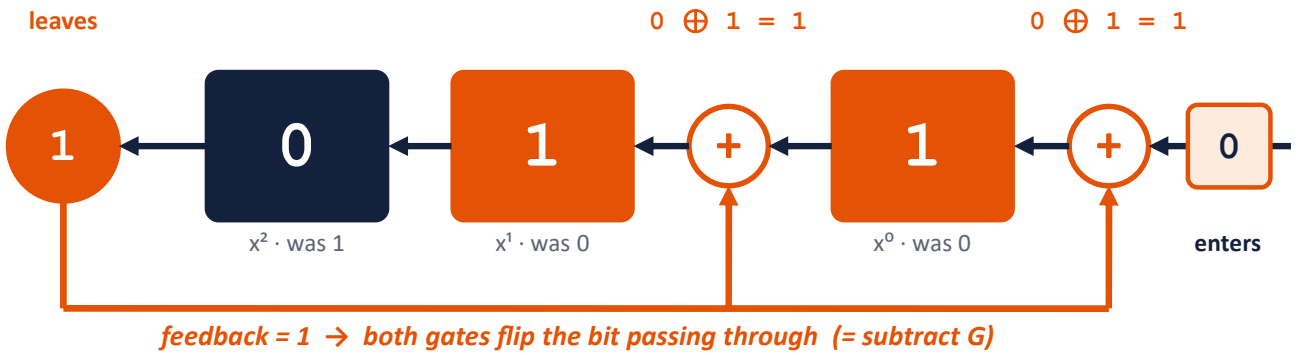
A 1 leaves again: window 1111 XOR 1011 = 0100.

EXAMPLE 1 IN THE CIRCUIT · TICK 7 OF 9

Tick 7 = long-division step 4

stream: 1 1 0 1 0 1 **0** 0 0

teal = data 110101 · orange = the three appended 0s



Same as step 4 of the table

window	1000
leading bit	1
XOR with	1011
result	0011

last 3 bits 011 = **register** ✓

REGISTER AFTER EACH TICK

tick 1	tick 2	tick 3	tick 4	tick 5	tick 6	**tick 7**	tick 8	tick 9
001	011	110	110	111	100	**011**		

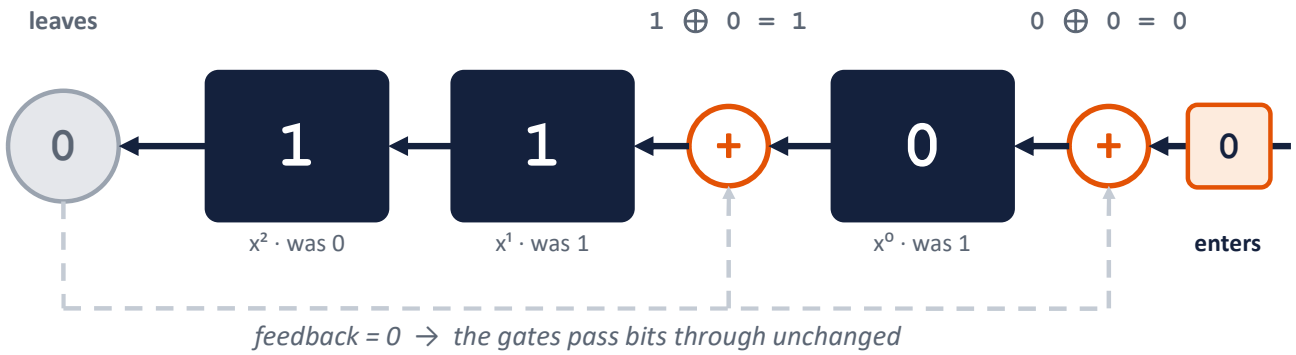
A 1 leaves: window 1000 XOR 1011 = 0011. The last data bit is in; now come the appended zeros.

EXAMPLE 1 IN THE CIRCUIT · TICK 8 OF 9

Tick 8 = long-division step 5

stream: 1 1 0 1 0 1 0 **0** 0

teal = data 110101 · orange = the three appended 0s



Same as step 5 of the table

window	0110
leading bit	0
XOR with	0000
result	0110

last 3 bits `110` = **register ✓**

REGISTER AFTER EACH TICK

tick 1	tick 2	tick 3	tick 4	tick 5	tick 6	tick 7	**tick 8**	tick 9
001	011	110	110	111	100	011	**110**	

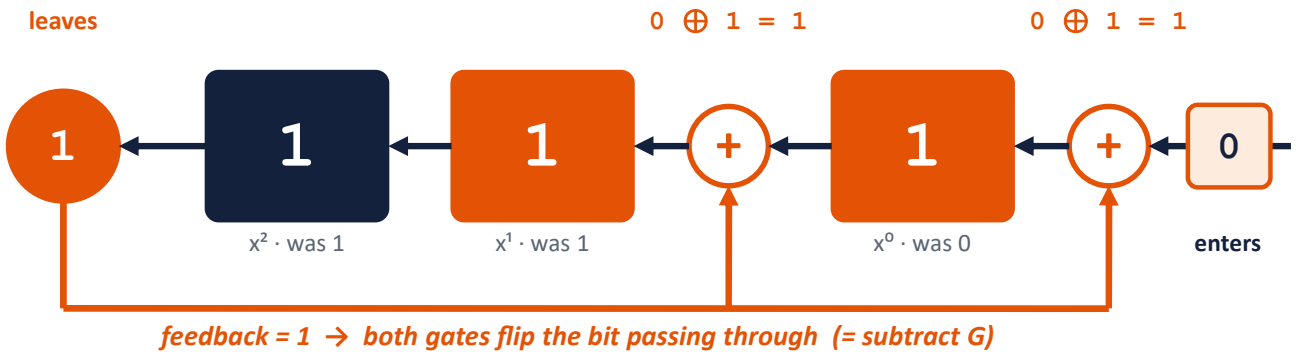
A 0 leaves, so this is the "XOR with 0000" step — the gates pass everything through.

EXAMPLE 1 IN THE CIRCUIT · TICK 9 OF 9

Tick 9 = long-division step 6

stream: 1 1 0 1 0 1 0 0 **0**

teal = data 110101 · orange = the three appended 0s



Same as step 6 of the table

window	1100
leading bit	1
XOR with	1011
result	0111

last 3 bits 111 = **register ✓**

REGISTER AFTER EACH TICK

tick 1	tick 2	tick 3	tick 4	tick 5	tick 6	tick 7	tick 8	tick 9
001	011	110	110	111	100	011	110	111

The final 1 leaves: window 1100 XOR 1011 = 0111. The register holds 111 — that is R.

After 9 ticks: R = 111

The same remainder as the long division — without ever writing the division out

Tick	Bit in	Bit out	Register	Division step
1	1	0	001	filling
2	1	0	011	filling
3	0	0	110	filling
4	1	1	110	step 1
5	0	1	111	step 2
6	1	1	100	step 3
7	0	1	011	step 4
8	0	0	110	step 5
9	0	1	111	step 6



The receiver uses the same circuit

Feed it the whole received frame, 110101111, and after 9 ticks it holds 000 — the check from earlier in this clip.



CRC-32 is the same picture

32 cells, with a gate wherever the CRC-32 generator has a 1.



Faster still

Real network cards handle 8, 32 or 64 bits per tick with wider logic — the result is identical.

On paper: rows of XOR, after the data has arrived. In hardware: one tick per bit, while the bits are still arriving. Same answer.

Generators in real life

Bit strings can be written as polynomials

$G = 1011$

1	0	1	1
x^3	x^2	x^1	x^0

$= x^3 + x + 1$

Each 1 is a term in the polynomial. Standards write generators this way.

Standard	r	Where you meet it
CRC-16	16	USB data packets
CRC-32	**32**	**Ethernet and Wi-Fi frames, ZIP files, PNG images**

CRC-32 uses a 33-bit generator.

You never invent your own generator. Good generators are chosen by experts and fixed in the standard, so every device agrees.

Why the link layer loves CRC

Strong against the errors real links produce, and fast in hardware

all bursts

of r bits or fewer are always detected (32 bits for CRC-32)

1 in 4 billion

roughly the chance a longer burst slips past CRC-32 (about 1 in 2^{32})

1 clock tick

per bit in hardware, with a few XOR gates and no buffering

Remember Clip 1: noise often hits many bits in a row. Parity and checksums handle bursts poorly; CRC was designed for them.

Four methods side by side

What each one costs and catches

Method	Extra bits	Detects	Can fix?	Where used
Parity	1	Any odd number of flipped bits	No	Simple serial links, older memory
2D parity	One per row and column	All 1-, 2-, and 3-bit errors	Yes, a single bit	Teaching; the idea behind real FEC codes
Checksum	16	Many errors, but not reordering or canceling	No	IPv4, UDP, TCP (software)
CRC-32	32	All bursts up to 32 bits, almost everything else	No	Ethernet, Wi-Fi (hardware)

Exam tip: be ready to explain why the link layer picks CRC while Layers 3-4 use a checksum.

An error was detected. Now what?

It depends on the link and on the layers above



Ethernet

Drops the frame silently. No retransmission at Layer 2; errors on cables are rare.



Wi-Fi

The receiver sends no acknowledgment, so the sender retransmits the frame right away.



TCP at the endpoints

Notices the missing data and resends it, end to end.



Live video over UDP

Nobody resends. You see a short glitch and the stream moves on.

Link to Class 1: this is the "reliable delivery" service. Wireless links provide it because they lose frames often.

Class 2: six takeaways

If you remember only one slide, remember this one

1	**Framing first**	The receiver must find where a frame starts and ends before it can check anything.
2	**Stuffing**	A flag needs escaping, because the flag pattern could appear inside the data.
3	**Redundancy**	Error detection adds extra bits computed from the data.
4	**Parity**	One bit catches an odd number of errors. 2D parity can also fix one bit.
5	**Checksum**	One's complement sum, flipped. Fast in software; used by IP, UDP, TCP.
6	**CRC**	Append r zeros, divide by G with XOR, send the remainder. Receiver expects 0.

TWO THINGS TO HAND IN BEFORE CLASS 3

Assignment: six quick questions

Q1 The payload is 7E 41 7D. What does the sender transmit between the flags?

Q2 What is the even parity bit for 1001 1101?

Q3 The 2D parity grid on the right has one flipped bit. Give its row, column, and correct value.

Q4 Compute the 8-bit checksum of 1010 1100 and 0111 0001.

Q5 D = 101101 and G = 1101. Compute the CRC remainder R.

Q6 In two or three sentences: why does Ethernet use a CRC instead of a checksum?

Grid for Q3

1	0	1	1	1
0	1	1	1	0
1	1	1	0	1
0	0	1	1	0

Orange cells are parity bits.

NEXT: CLASS 3 · LIVE IN THE LECTURE ROOM

Many trucks, one narrow road

Multiple access, Ethernet, switches, VLANs, and Wi-Fi. You will become the network.

- **The Talking Game**
- **The Human Switch**
- **Bring your phone**



Think before class

In a group chat, what happens when two people talk at the same time? How do humans fix it?